

19 transacciones de historia + 1 que se evalúa

Riesgo de fraude transaccional en producción

Modelo B (GRU) del Proyecto 1 como servicio web

Jorge López · Angel Herrarte

Deep Learning y Sistemas Inteligentes · Universidad del Valle · noviembre 2026

¿Aporta el orden de las transacciones?

Jorge

1 Problema

Un motor antifraude usa agregados (montos, frecuencia, ciudades). ¿La secuencia aporta información adicional y reduce el costo de errores?

2 Usuario

Analista de fraude que prioriza revisiones. Opcional: motor de autorización que usa el puntaje como señal extra.

3 Decisión

Revisar, permitir o bloquear según su propia política. El servicio entrega puntaje y versión; no bloquea.

Q4,200

costo por fraude no detectado (FN)

Q180

costo por legítima bloqueada (FP)

Costos del caso simulado del Proyecto 1, no de un banco real.

Planificación: 11 al 15 de noviembre

Jorge

11 nov

Jorge

Preparación causal, reentrenamiento de A, B y C, cargador CPU, API y reproducción limpia (134 pruebas)

13 nov

Angel

Interfaz web, imagen Docker verificada en Linux, configuración de Render y monitoreo

14 nov

Angel · ambos

Publicación, pruebas desde otro dispositivo, diapositivas y revisión conjunta

15 nov

Ambos

Ensayo, revisión de enlaces y entrega antes de las 23:59

La interfaz y el despliegue estaban planificados para el 11 y 12 de noviembre; se completaron el 13. Detalle por etapa en la sección 2 del README.

Supuestos sobre datos, usuario y entorno

Jorge

Mecanismo	Patrón simulado	Disponible en
F1	Compras pequeñas en rampa y un cobro grande	Todas las particiones
F2	Ciudades lejanas con poca separación temporal	Todas las particiones
F3	Ráfaga de ecommerce caro de madrugada	Todas las particiones
F4	Goteo lento: cargos pequeños recurrentes	**Solo prueba**

Además, un señuelo legítimo parecido a F1 para comprobar que el modelo no solo reacciona al monto.

- Datos sintéticos: 6,000 tarjetas, 180 días, 484,738 transacciones.
- El consumidor envía 1 a 20 eventos de una tarjeta, en orden; sin etiquetas.
- El servicio corre en CPU y no guarda historial.
- Tiempo en días desde un origen fijo; fechas reales requieren conversión.

Modelo B: GRU sobre la secuencia de eventos

Jorge

20 eventos
rubro, canal, ciudad + 7 numéricas

Embeddings
3 × dimensión 8

GRU
1 capa, estado 64

Cabeza
64 → 32 → 1, dropout 0.2

Puntaje
sigmoide, 0 a 1

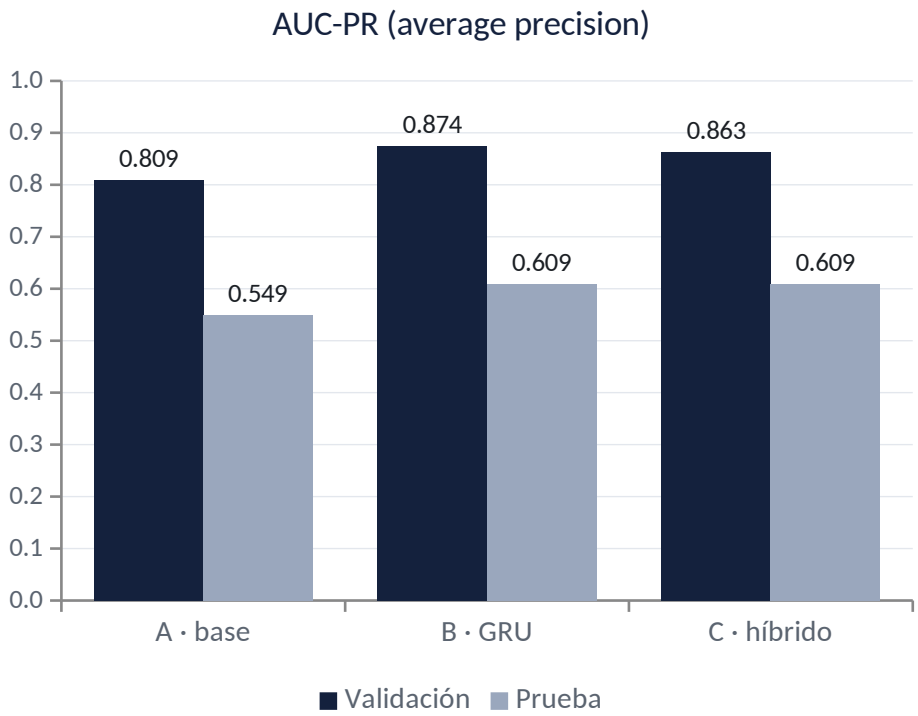
Cambio justificado

La ciudad base se calculaba con toda la historia de la tarjeta, incluido el futuro. Ahora usa solo las 19 transacciones previas. Se reentrenaron A, B y C con la misma preparación, sin tocar arquitectura ni épocas.

Modelo	Entrada
A · línea base	Boosting con 15 agregados de la ventana
B · servicio	Secuencia de eventos (GRU)
C · híbrido	GRU + los 15 agregados

Hipótesis frente a la línea base

Jorge



✓

B supera a A por +0.065 en validación

La secuencia aporta frente a agregados de la misma ventana.

✗

C: pedía +0.020 sobre B; quedó -0.011

Criterio fijado antes de entrenar. La hipótesis de C no se cumplió.

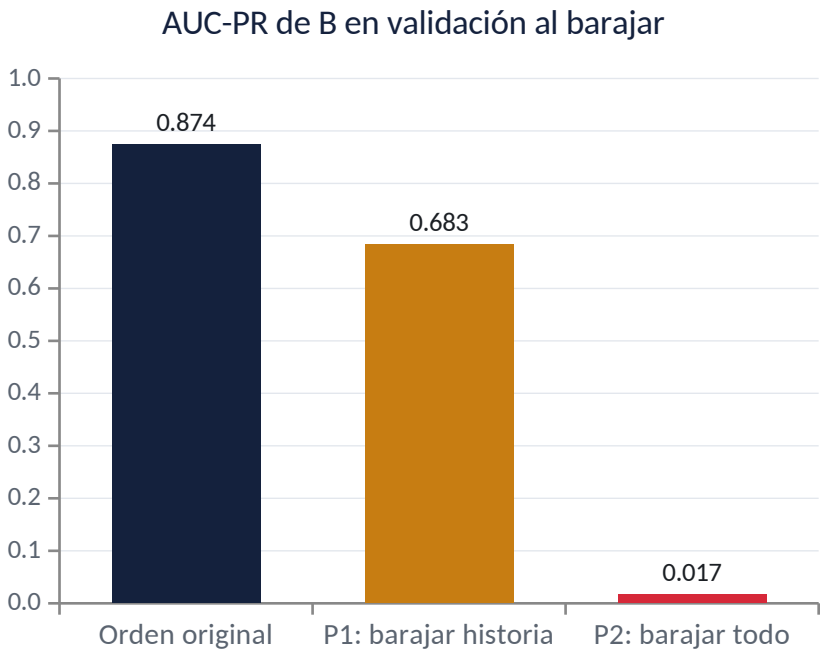
=

Se mantiene B en el servicio

C rinde algo mejor en prueba, pero no se elige mirando prueba.

B usa el orden, y reduce falsos positivos

Jorge



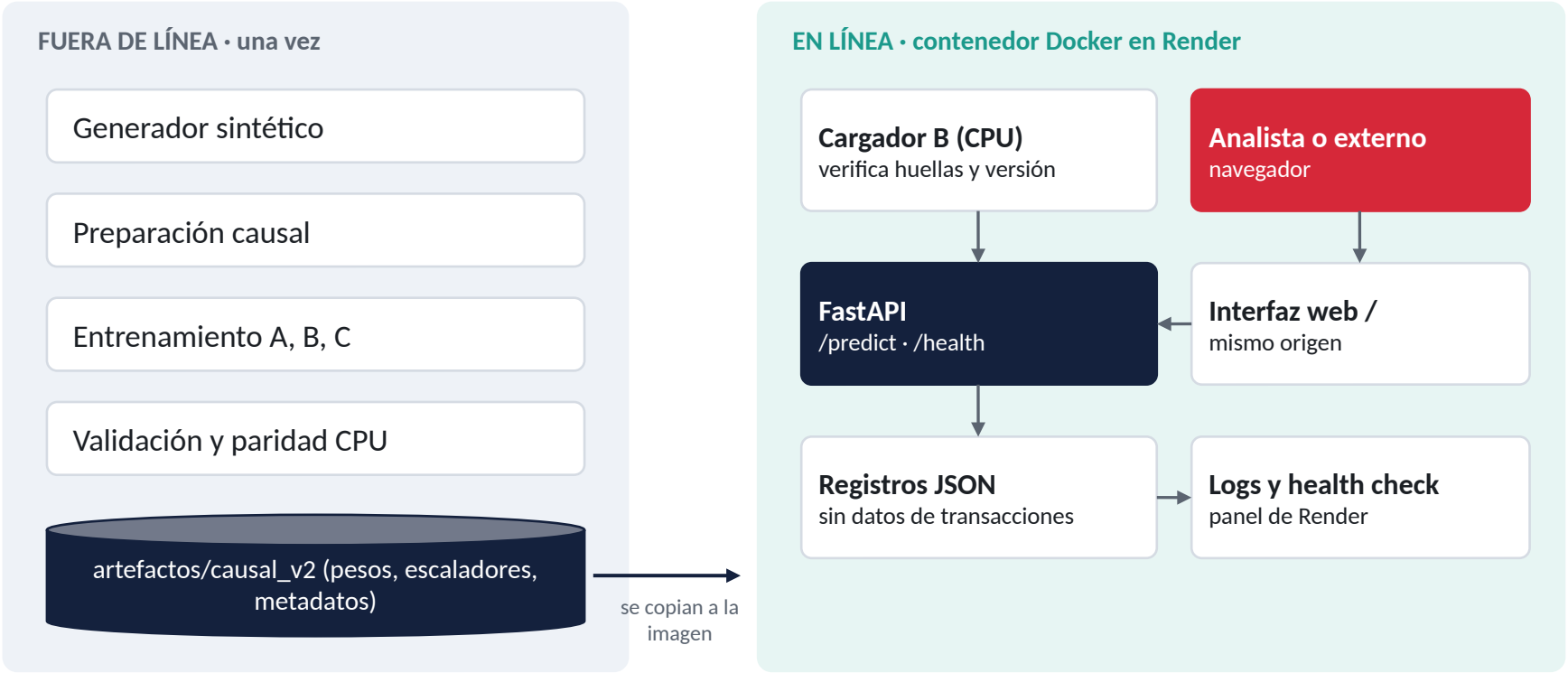
Prueba	Precisión	Exhaust.	FP	FN	Costo
A	0.34	0.67	803	204	Q1,001,340
B	**0.53**	**0.67**	**371**	**203**	**Q919,380**
C	0.48	0.69	457	193	Q892,860

- B reduce falsos positivos de 803 a 371 frente a A con exhaustividad similar.
- F2 y F3: exhaustividad 1.0. F4: 0.0 en los tres modelos.

Umbral elegido en validación minimizando 4200·FN + 180·FP. Costos simulados.

Arquitectura: entrenar fuera, servir en línea

Angel



Demo: una persona externa obtiene puntaje

Angel





- Elegir un ejemplo o pegar JSON (1 a 20 eventos).
- Calcular: puntaje, umbral de referencia, versión e ID de solicitud.
- Errores claros: un monto negativo marca la celda (422).

Probar en vivo: https://riesgo-transaccional.onrender.com

Evidencias del pase a producción

Angel

6/6

casos idénticos en Render (tolerancia 1e-6)

280 ms

mediana de latencia desde el cliente

140

pruebas automáticas aprobadas

356 MiB

RAM en Docker local; plan de 512 MB

Evidencia	Dónde
Servicio público e interfaz	https://riesgo-transaccional.onrender.com
Verificación contra los seis casos	evidencias/verificacion_render.json
Registros JSON por solicitud	evidencias/ y logs de Render
Capturas de uso	evidencias/capturas/
Suite de pruebas y reproducción limpia	Proyecto1/tests · docs/REPRODUCCION_LIMPIA.md

Operación y monitoreo

Angel

✓ Implementado

- Salud: GET /health y health check de Render.
- Registro JSON por solicitud: fecha, ID, ruta, estado, duración y versión.
- Errores: filtrar estado 422 y 5xx en los logs.
- Latencia: duracion_ms en servidor y verificador desde el cliente.

→ Propuesto

- Deriva de datos: agregados diarios anónimos y PSI; PSI > 0.2 sostenido = revisar.
- Desempeño: recalcular AUC-PR con contracargos cada mes.
- Recalibrar el umbral si cambian prevalencia o costos.
- Reentrenar ante nuevos mecanismos (F4) con la misma validación causal.

Límites y recomendación al comité

Angel

Límites

- F4 (goteo lento): exhaustividad 0 en los tres modelos.
- F1 puede pasar inadvertido; la demo muestra un caso.
- Datos y costos sintéticos: no se trasladan a un banco.
- Puntaje no calibrado; prueba ya vista en Proyecto 1.

Recomendación

Mantener B en producción como demostración y señal de priorización, **no como bloqueo automático.**

Antes de un uso real: validar con datos reales, calibrar el puntaje e implementar el monitoreo de deriva.



Gracias. ¿Preguntas?

Demo: https://riesgo-transaccional.onrender.com

Repositorio: github.com/Jorge162017/P1-DeepLearning

Jorge López · Angel Herrarte